

Gestell Doxygen Documentation Rules for Embedded C

1. File Documentation (@file, @brief, @details, @version, @date, @author, @copyright)

Every .c and .h file must start with a header block containing metadata.

- **@file**: Exact file name (e.g., `uart_driver.h`).
- **@brief**: Short, one-sentence summary of the file.
- **@details**: Extended description of driver, peripheral, or logic.
- **@version**: Release version (e.g., `1.0.0`).
- **@date**: Creation or last modification date.
- **@author**: Developer name and email contact.
- **@copyright**: Copyright ownership statement.

Example:

```
C/C++
/**
 * @file      uart_driver.h
 * @brief     UART driver interface.
 * @details   Provides initialization, configuration, and communication APIs
for UART.
 * @version   1.0.0
 * @author    Hesham Ahmed <Hisham.ah.hamed@gmail.com>
 * @date      2026-09-23
 * @copyright Copyright (c) 2026 Gestell. All rights reserved.
 */
```

2. Function Documentation (@fn, @brief, @details, @param, @return, @retval, @warning, @note)

Every public API function prototype must be documented in header files.

- **@fn**: Explicit tag for function name.
- **@brief**: Concise summary starting with an active verb (e.g., "Initializes", "Sends").
- **@details**: Extended explanation of function operation, state machine, or hardware behavior.
- **@param[in]**: Input parameter passed to the function.
- **@param[out]**: Output parameter (e.g., pointer/buffer populated by the function).
- **@param[in,out]**: Parameter acting as both input and output.
- **@return**: Description of return type and general status.
- **@retval**: Specific return values and their conditions (e.g., **0** for success, **-1** for error).
- **@warning**: Critical constraints or risks (e.g., mandatory execution sequence).
- **@note**: Operational or implementation notes.

Example:

```
C/C++
/**
 * @fn      UART_Send
 * @brief   Sends a stream of data over UART.
 * @details Updates the TX buffer with data and blocks until transmission is
complete.
 * @param[in] data    Pointer to the data buffer to be sent.
 * @param[in] length  Number of bytes to transmit.
 * @return   Execution status of the transmission operation.
 * @retval   0  Transmission succeeded.
 * @retval   -1 Transmission failed or invalid parameters.
 * @warning  UART_Init() must be called before using this function.
 */
int UART_Send(const uint8_t *data, uint16_t length);
```

3. Macros & Constants (@def, @brief, @param)

Document preprocessor directives, macros, and bit-manipulation functions using **@def**.

- **@def**: Explicit tag specifying the macro name.
- **@brief**: Short summary of the constant or macro behavior.
- **@param**: Description of parameters for function-like macros.

Example:

```
C/C++
/**
 * @def    MAX_BUFFER_SIZE
 * @brief Defines the maximum buffer size (in bytes) for UART transmission.
 */
#define MAX_BUFFER_SIZE 256

/**
 * @def    SET_BIT
 * @brief Sets a specific bit in a target hardware register.
 * @param REG Target 32-bit register address.
 * @param BIT Bit position (0-31) to set.
 */
#define SET_BIT(REG, BIT) ((REG) |= (1U << (BIT)))
```

4. Enumerations (@enum, @var, Inline Syntax)

Document enumerations using @enum and individual members using @var or inline comments /**< ... */.

Option A: Explicit Tags (@enum & @var)

```
C/C++
/**
 * @enum Status_t
 * @brief System operation status codes.
 */
typedef enum {
    /** @var Status_t::STATUS_SUCCESS
     * @brief Operation completed successfully.
     */
    STATUS_SUCCESS = 0,

    /** @var Status_t::STATUS_ERROR
     * @brief Operation failed due to general error.
     */
    STATUS_ERROR = 1
} Status_t;
```

Option B: Inline Member Documentation (Recommended)

```
C/C++
/**
 * @brief System operation status codes.
 */
typedef enum {
    STATUS_SUCCESS = 0, /**&lt; Operation completed successfully */
    STATUS_ERROR   = 1  /**&lt; Operation failed due to general error */
} Status_t;
```

5. Structures & Unions (@struct, @union, @var, Inline Syntax)

Document structures (struct) and unions (union) using explicit tags or inline comments.

Option A: Explicit Tags (@struct / @union & @var)

```
C/C++
/**
 * @struct TimerConfig_t
 * @brief Timer peripheral initialization parameters.
 */
typedef struct {
    /** @var TimerConfig_t::prescaler
     * @brief Clock prescaler divider value.
     */
    uint16_t prescaler;

    /** @var TimerConfig_t::period
     * @brief Timer auto-reload period value.
     */
    uint16_t period;
} TimerConfig_t;
```

Option B: Inline Member Documentation (Recommended)

```
C/C++
/**
 * @brief Timer peripheral initialization parameters.
 */
typedef struct {
    uint16_t prescaler; /**&lt; Clock prescaler divider value */
    uint16_t period;    /**&lt; Timer auto-reload period value */
} TimerConfig_t;
```

6. Groups & Modules (@defgroup, @addtogroup, @{, @})

Organize related APIs, macros, enums, or constants into logical groups in generated documentation.

- **@defgroup**: Defines a new group identifier and display title.
- **@addtogroup**: Adds entities to an existing group.
- **@{** and **@}**: Opens and closes the group block.

Example: Grouping System Error Codes

```
C/C++
/**
 * @defgroup Error_Codes System Error Codes
 * @brief Standard error codes returned by system and driver functions.
 * @{
 */

#define ERROR_NONE      0 /**&lt; No error occurred */
#define ERROR_TIMEOUT  1 /**&lt; Operation timed out */
#define ERROR_BUSY     2 /**&lt; Peripheral resource is busy */
#define ERROR_INVALID  3 /**&lt; Invalid parameter supplied */

/** @} */ // End of Error_Codes group
```

7. Special Doxygen Tags (@todo, @deprecated, @bug, @see, @ref, @code, @endcode)

- **@todo**: Marks pending tasks or future enhancements.
- **@deprecated**: Identifies legacy APIs that should no longer be used.
- **@bug**: Documents known bugs or hardware limitations.
- **@see**: Cross-references other functions, files, or documentation sections.
- **@ref**: Creates an inline hyperlink to a documented symbol.
- **@code ... @endcode**: Embeds C code examples inside documentation blocks.

Example:

C/C++

```
/**
 * @brief   Reads data from the UART RX buffer.
 * @see     UART_Send
 * @note    Non-blocking function call.
 * @todo    Implement DMA support in v2.0.0.
 * @bug     Buffer overflow may occur if baud rate exceeds 115200 without
hardware flow control.
 *
 * @code
 * uint8_t rx_data;
 * if (UART_Read(&rx_data) == STATUS_SUCCESS) {
 *     // Process received byte
 * }
 * @endcode
 */
int UART_Read(uint8_t *data);
```

8. General Best Practices

1. **Conciseness**: Keep @brief descriptions short and focused.
2. **Consistent Naming**: Follow standard prefix rules (e.g., **GPIO_Init**, **UART_Send**, **SPI_Receive**).
3. **Consumer Perspective**: Focus on how the caller uses the driver API rather than internal driver register details.
4. **Data Direction**: Always specify **@param[in]**, **@param[out]**, or **@param[in,out]** for pointers in Embedded C.

9. Multi-Language Support & Syntax Differences (C++, Java, C#, Python)

While core Doxygen tags (@brief, @param, @return, @see, @note) remain consistent across programming languages, comment block syntaxes and object-oriented tags vary based on language features.

A. Comment Block Syntaxes by Language

- **C / C++ / Java / JavaScript:**
 - Uses block comments `/** ... */` or triple slashes `///`.
- **C#:**
 - Uses triple slashes `///` or `/** ... */`. Doxygen also parses native C# XML documentation comments (`<summary>`, `<param>`, etc.).
- **Python:**
 - Uses double hash `##` comments or native triple-quoted docstrings `""" ... """`.
 - *Note: In Doxyfile, set `PYTHON_DOCSTRING = YES` to enable native Python docstring parsing.*

B. Language-Specific Doxygen Tags

1. **C++ (Classes, Namespaces & Templates):**
 - `@class`: Documents C++ classes (`@class Controller`).
 - `@namespace`: Documents C++ namespaces (`@namespace Drivers`).
 - `@tparam`: Documents template parameters (`@tparam T Type of data element`).
 - `@public`, `@protected`, `@private`: Specifies member access levels (automatically detected by Doxygen in C++ headers).
2. **Java & C# (OOP & Exceptions):**
 - `@package`: Documents Java package or C# namespace.
 - `@throws` / `@exception`: Documents exceptions thrown by methods (`@throws IOException If connection fails`).
 - `@interface`: Documents interface contracts (`@interface ICommunicationDriver`).
3. **Python (Modules & Docstrings):**
 - `@package`: Documents Python module/package.
 - Doxygen automatically extracts function signature type hints (`def process(data: str) -> bool`).

C. Doxyfile Configuration Parameters per Language

Adjust settings in Doxyfile based on the primary language:

- **Embedded C:** OPTIMIZE_OUTPUT_FOR_C = YES
- **C++:** OPTIMIZE_OUTPUT_FOR_C = NO (*Default*)
- **Java / C#:** OPTIMIZE_OUTPUT_JAVA = YES
- **Python:** OPTIMIZE_OUTPUT_JAVA = YES, EXTENSION_MAPPING = py=Python